Relationship Scores / Genesis OS

Missing Data Layers Engineering Blueprint

AI feedback loops • emotional data schemas • event logging • API contracts • prediction engine • longitudinal growth system

Purpose	Close every major system gap before full-scale development so engineering can build without guessing.
Audience	Founders, product architects, full-stack engineers, backend/API developers, AI engineers, data engineers, QA, and analytics.
Scope	Structured emotional logging, behavior tracking, perception vs reality, attachment triggers, forecast models, exact schemas, contracts, workflows, and failure handling.
Build principle	Everything important becomes a defined object, event, or metric. If it matters, it must be tracked, versioned, and queryable.

1. Executive Summary

This document defines the missing infrastructure required to turn Relationship Scores from a strong scoring system into a true emotional intelligence platform. The goal is not merely to calculate compatibility. The goal is to create a living behavioral graph that captures what a user says, how they feel, what they actually do, how those patterns change over time, and how the AI layer uses that stream to forecast outcomes and improve guidance.

What this blueprint adds:

- A full structured emotional data model for daily state, post-interaction state, post-conflict state, silence response, and longitudinal trendlines.
- Behavioral event logging for initiation, response latency, tone movement, withdrawal, escalation, repair, avoidance, and pattern recurrence.
- Perception-vs-reality architecture that compares self-report with objective interaction data and detects self-awareness gaps.
- Attachment trigger capture that records what activates anxiety, shutdown, overpursuit, defensiveness, and distancing behaviors.
- A standardized emotional data packet that all AI services consume so scoring, insights, forecasting, and analytics read from the same payload.
- Clear API contracts, event schemas, workflow steps, and edge-case handling so engineering can build deterministically.
- A forecast layer that predicts relationship risk, likely trigger cycles, repair probability, silence deterioration, and growth potential.

Core design rule:

Every meaningful emotional signal must exist in one of three forms: a durable profile attribute, a timestamped event, or a computed metric. If it does not exist in one of those forms, the AI cannot trust it, the analytics layer cannot use it, and the product cannot learn from it.

2. System Design Principles

No.	Principle	Meaning in build
1	Event-sourced truth	The system stores raw events first, then derives aggregates and AI insights from those events. This prevents loss of detail and lets models improve later.
2	Objective + subjective pairing	Important emotional moments should capture both self-report and actual behavior. One without the other creates blind spots.

3	Version everything important	Scores, archetypes, shadows, trigger maps, forecast outputs, and user states must be time-versioned to enable growth tracking.
4	Consent-gated observation	Any chat analysis, behavioral inference, or couple forecasting must obey explicit consent states and feature scope flags.
5	Separation of raw data from inference	Do not overwrite source data with AI interpretations. Store AI interpretations as separate versioned objects.
6	Graceful degradation	The system must continue functioning if AI is down, if one partner has not consented, or if only partial data exists.

3. The Five Missing Data Layers and How to Build Them

3.1 State-Based Emotional Tracking

- Purpose: capture how a user feels at meaningful checkpoints, not just who they tested as during onboarding.
- Moments to log: daily check-in, before conversation, after conversation, after conflict, after silence period, after reconnection, after date, after Sanctuary ritual.
- Minimum state fields: valence, arousal/intensity, emotional clarity, felt safety, felt closeness, felt rejection, hope, exhaustion, confusion, desire to pursue, desire to withdraw, regulation level.
- Store both scalar ratings and optional free-text so the system has structured inputs and human nuance.
- Compute trend lines over 7-day, 30-day, and 90-day windows.

3.2 Behavioral Pattern Logging

- Purpose: capture what people actually do in interaction loops.
- Track initiations, response latency bands, message burst patterns, silence duration, tone swing, escalation, de-escalation, apology, repair attempt, reassurance seeking, pursuit after withdrawal, conflict exits, and restart behavior.
- These are emitted as immutable events with actor, target, timestamp, context, and derived metrics.
- Use these events to compute behavioral signatures such as initiator ratio, average repair latency, volatility index, pursuit-withdrawal score, and conflict recovery score.

3.3 Perception vs Reality Layer

- Purpose: compare what a user believes happened with what the interaction data shows.

- After key exchanges, ask simple prompts such as: How connected did that feel, Did you feel ignored, Did that interaction calm you or activate you, and Do you think you were understood.
- Then compare these perceptions against event metrics such as response time, sentiment shift, interruption count, message balance, repair attempts, and tone stabilization.
- Output a self-awareness delta metric and category-specific distortion flags, never framed as moral judgment.

3.4 Attachment Activation Triggers

- Purpose: identify what reliably activates insecurity or distancing behavior.
- Trigger classes should include ambiguity, delayed reply, mixed signals, criticism, unmet expectation, perceived replacement, emotional intensity, pressure for definition, conflict, vulnerability exposure, and lack of reassurance.
- Each trigger log records trigger type, context, intensity, perceived meaning, body reaction, behavior that followed, and whether repair occurred.
- These records build each user's trigger map and later power forecasting, prompts, and compatibility translation.

3.5 Longitudinal Growth Tracking

- Purpose: show how a user changes over time instead of pretending the first assessment is the whole story.
- Version profile scores, archetype confidence, shadow confidence, trigger map, behavioral signatures, and forecast outputs.
- Create snapshots monthly, quarterly, after major relationship events, and after meaningful assessment retakes.
- Measure drift, stability, improvement, regression, and volatility.

4. Exact Data Schemas

Recommended storage pattern: PostgreSQL for transactional truth, JSONB only for flexible metadata, a warehouse or analytics layer for aggregate reporting, and a vector/index layer only for AI memory artifacts that need retrieval. Do not hide core business data inside opaque JSON blobs.

user_emotional_state

Field	Type	Meaning
id	uuid	Primary key
user_id	uuid	Owner user
context_type	enum	daily_checkin, pre_chat, post_chat, post_conflict, post_silence, post_date, post_ritual
context_ref_id	uuid nullable	Conversation, conflict episode, ritual session, or relationship reference

state_at	timestamptz	When the state was recorded
valence_score	smallint	-5 to +5 overall emotional tone
intensity_score	smallint	0 to 10 intensity
clarity_score	smallint	0 to 10
felt_safety_score	smallint	0 to 10
felt_closeness_score	smallint	0 to 10
felt_rejection_score	smallint	0 to 10
hope_score	smallint	0 to 10
urge_to_pursue_score	smallint	0 to 10
urge_to_withdraw_score	smallint	0 to 10
regulation_score	smallint	0 to 10
primary_emotions	text[]	Controlled list, max 3
free_text	text nullable	Optional user note
source	enum	user_manual, prompted_checkin, ai_inferred
confidence_score	numeric(5,2)	AI inference confidence; 1.00 for user manual
created_at	timestamptz	Audit

behavior_event

Field	Type	Meaning
id	uuid	Primary key
relationship_id	uuid nullable	Couple or interaction context
conversation_id	uuid nullable	Conversation context
actor_user_id	uuid	Who did the action
target_user_id	uuid nullable	Who the action affected
event_type	enum	message_sent, conversation_started, reply_delay_band, tone_shift, withdrawal_start, withdrawal_end, escalation_start, escalation_end, apology, repair_attempt,

		reassurance_request, reconnection
event_at	timestamptz	When it happened
sequence_index	bigint	Ordering within conversation stream
payload	jsonb	Event-specific details
derived_metrics	jsonb	Latency, sentiment delta, interruption flags, etc.
source	enum	system_observed, ai_classified, user_reported
created_at	timestamptz	Audit

perception_check

Field	Type	Meaning
id	uuid	Primary key
user_id	uuid	Reporter
context_ref_id	uuid	Conversation or conflict episode
context_type	enum	conversation_segment, conflict_episode, silence_period, date_event
submitted_at	timestamptz	When submitted
felt_heard_score	smallint	0 to 10
felt_connected_score	smallint	0 to 10
felt_ignored_score	smallint	0 to 10
felt_misunderstood_score	smallint	0 to 10
felt_calm_after_score	smallint	0 to 10
summary_text	text nullable	Optional note
created_at	timestamptz	Audit

trigger_activation

Field	Type	Meaning
id	uuid	Primary key
user_id	uuid	Whose trigger activated

relationship_id	uuid nullable	Context
trigger_type	enum	delay, ambiguity, criticism, mixed_signal, pressure, vulnerability, perceived_replacement, unmet_expectation, conflict, withdrawal
trigger_at	timestamptz	Timestamp
context_ref_id	uuid nullable	Linked episode
intensity_score	smallint	0 to 10
perceived_meaning	text nullable	What user thought it meant
body_signal_tags	text[]	heart_race, numbness, stomach_drop, anger_heat, etc.
followup_behavior_tags	text[]	double_text, detach, shutdown, argue, overexplain, test, disappear
repair_occurred	boolean	Whether repair later happened
resolved_at	timestamptz nullable	Optional
created_at	timestamptz	Audit

profile_snapshot

Field	Type	Meaning
id	uuid	Primary key
user_id	uuid	Owner
snapshot_type	enum	monthly, quarterly, post_assessment, post_relationship_change, ad_hoc
effective_at	timestamptz	Snapshot timestamp
readiness_score	numeric(5,2)	0-100
primary_archetype_id	text	Current top archetype
primary_archetype_confidence	numeric(5,2)	0-1
primary_shadow_id	text nullable	Current top shadow
primary_shadow_confidence	numeric(5,2)	0-1

behavior_signature	jsonb	Derived behavior metrics
trigger_map_summary	jsonb	Top triggers and intensities
growth_metrics	jsonb	Trend deltas
forecast_summary	jsonb	Latest prediction summary
created_at	timestamptz	Audit

ai_insight_record

Field	Type	Meaning
id	uuid	Primary key
subject_type	enum	user, relationship, conversation, conflict_episode
subject_id	uuid	Target entity
insight_type	enum	state_summary, behavior_summary, trigger_pattern, self_awareness_delta, forecast, intervention_suggestion
insight_version	int	Version counter
input_packet_id	uuid	Which emotional packet created it
model_name	text	Model identifier
output_json	jsonb	Structured insight
narrative_text	text nullable	Optional human-readable summary
confidence_score	numeric(5,2)	0-1
status	enum	active, superseded, failed_review
created_at	timestamptz	Audit

forecast_record

Field	Type	Meaning
id	uuid	Primary key
subject_type	enum	pair, user_in_relationship
subject_id	uuid	Target
forecast_window_days	int	7, 30, 90

forecast_type	enum	conflict_risk, silence_decay, repair_probability, rupture_probability, growth_probability, attachment_activation
score	numeric(5,2)	0-100 risk/probability
band	enum	low, moderate, high, severe
drivers_json	jsonb	Top variables that drove output
recommended_actions_json	jsonb	Interventions or prompts
packet_id	uuid	Source packet
model_version	text	Version
created_at	timestamptz	Audit

5. Standardized Emotional Data Packet

All AI services should consume one normalized packet format. This prevents every model or microservice from inventing its own input structure. The packet is a point-in-time summary composed from raw events, states, profile attributes, and relationship context.

- Packet levels: user packet, relationship packet, conversation packet, conflict packet, and silence-period packet.
- Packets are generated on demand for AI calls and optionally persisted for traceability.
- Every packet must include packet_id, subject scope, time window, consent flags, source completeness score, and freshness timestamp.

Packet block	Contents
packet_id	UUID for traceability and audit
packet_type	user_daily, relationship_30d, conflict_episode, conversation_segment, silence_period
subject	User or pair identifiers
time_window	Start/end timestamps and rolling period
consent_scope	What data AI is authorized to read
profile_summary	Current archetype, shadow, readiness, preferences, demographics if allowed
latest_state_summary	Most recent structured emotional states and rolling averages
behavior_signature	Initiator ratio, response latency, volatility, repair behavior, pursuit-withdrawal pattern

recent_event_rollup	Counts and important sequence markers from recent events
trigger_map	Top triggers, average intensity, most common follow-up responses
perception_reality_delta	Differences between self-report and objective interaction metrics
forecast_inputs	Features passed into forecast layer
relationship_context	Stage, exclusivity, relationship mode, known rupture markers
data_quality	Completeness, confidence, stale-field warnings

6. API Contracts

Recommendation: build internal service contracts as JSON-based REST endpoints first, even if you later expose GraphQL or gRPC. The first priority is deterministic engineering handoff, observability, and easy QA.

Endpoint	Purpose	Contract summary
POST /v1/emotional-states	Create structured emotional state entry	Input: user_id, context_type, scores, emotions, free_text. Output: state_id plus normalized record.
POST /v1/behavior-events	Ingest raw or AI-classified interaction event	Input: actor, target, event_type, event_at, payload. Output: event_id, ingestion status, derived metric preview.
POST /v1/perception-checks	Submit perception-vs-reality survey moment	Input: context_ref_id, feeling scores, summary_text. Output: record id and whether comparison job queued.
POST /v1/trigger-activations	Record attachment trigger activation	Input: trigger type, intensity, context, behavior tags. Output: record id and trigger-map recompute status.
GET /v1/users/{id}/growth-snapshot	Return latest longitudinal snapshot	Output: latest scores, archetype drift, shadow drift, behavior trend, trigger trend, forecast summary.
POST /v1/packets/build	Build standardized emotional packet	Input: subject scope, time window, packet type. Output: packet JSON plus completeness indicators.

POST /v1/ai/insights/generate	Generate structured AI insight from packet	Input: packet_id, insight_type. Output: ai_insight_record.
POST /v1/forecasts/generate	Generate forecast record	Input: packet_id, forecast_window_days, forecast_types. Output: one or more forecast records.
GET /v1/relationships/{id}/behavior-signature	Return derived relationship behavior metrics	Output: ratio, latencies, volatility, repair trends, silence stats.
POST /v1/jobs/recompute-profile	Recompute user aggregates after major changes	Input: user_id, reason. Output: async job id.

API response rules:

- Every write endpoint returns id, created_at, schema_version, and validation_errors if any.
- Every AI/forecast endpoint returns status, input_packet_id, model_version, confidence_score, and reason codes for missing inputs.
- Every aggregate endpoint returns data_freshness and recompute_pending flags.
- Use idempotency keys for event ingestion to prevent duplicate behavior records.

7. Event Tracking System

Every meaningful action should create a machine-readable event. This is the audit trail and the behavioral truth layer. Do not rely only on current-state tables.

Group	Canonical event names
Assessment events	assessment_started, question_answered, answer_changed, assessment_completed, retake_requested
Messaging events	conversation_opened, message_sent, message_read, first_reply_after_gap, conversation_started_by_user
Silence events	silence_window_started, silence_window_24h, silence_window_72h, silence_window_7d, reconnection_after_silence
Conflict events	conflict_flagged, escalation_start, escalation_peak, withdrawal_start, repair_attempt, apology_sent, conflict_resolved
Sanctuary events	checkin_submitted, stillness_started, whisper_created, archive_entry_saved, ritual_completed
Forecast events	forecast_requested, forecast_generated, intervention_prompt_displayed,

	intervention_completed

Required standard event fields:

- event_id, event_name, schema_version, occurred_at, actor_user_id, target_user_id if applicable, relationship_id if applicable, conversation_id if applicable, source, client_platform, request_id, idempotency_key, metadata_json.
- All event names must be documented in a tracking catalog so analytics, backend, and AI read the same definitions.

8. AI Feedback Loop

The AI layer should not be a loose prompt stapled onto the app. It should be a defined loop with controlled inputs, deterministic outputs, and measurable learning.

Step	What happens
1. Capture	User actions, state check-ins, message behavior, perception checks, and trigger logs create raw records.
2. Normalize	Event workers validate schema, enrich records, derive initial metrics, and store in canonical tables.
3. Aggregate	Materialized views or jobs compute rolling behavior signatures, trigger intensity averages, and trend deltas.
4. Build packet	A packet builder composes the latest relevant profile + state + behavior + context + quality metadata.
5. Generate insight	AI service reads packet and produces structured JSON first, optional narrative second.
6. Forecast	Forecast service consumes the same packet and outputs risk/probability records.
7. Intervene	Product surfaces reflection prompts, Sanctuary nudges, pacing suggestions, or couple repair tools.
8. Observe outcome	System logs whether user ignored, completed, or benefited from intervention.
9. Learn	Analytics compares forecast vs outcome and intervention vs outcome to recalibrate rules or models.

What gets learned:

- Which triggers most often precede withdrawal, escalation, or rupture.
- Which intervention prompts increase repair, calm, or reconnection probability.
- How forecast confidence changes as more behavioral data accumulates.
- Which self-reports are consistently misaligned with observed patterns.
- How archetype or shadow confidence should update when behavior contradicts initial assessment.

9. Prediction Layer

Forecasting should begin with transparent rules + weighted features, then evolve into model-assisted prediction once enough labeled outcome data exists.

Initial forecast products:

- Conflict risk in next 7/30 days
- Silence deterioration risk
- Repair probability after rupture
- Attachment activation probability by trigger type
- Growth probability if interventions are followed
- Stability score for new matches with limited observed data

Starter feature set for forecasting:

- Rolling emotional valence and volatility
- Average response delay and delay variability
- Initiator ratio imbalance
- Escalation frequency and duration
- Repair attempt rate and average repair latency
- Perception-reality delta trend
- Trigger activation frequency and mean intensity
- Readiness score, archetype fit signals, shadow indicators
- Silence windows and reconnection success rate
- Assessment contradiction cluster count

Forecast labels to collect for later supervised learning:

- Did conflict occur in window
- Did silence exceed threshold
- Did users reconcile within threshold
- Did relationship status deteriorate, stabilize, or improve
- Did user report greater felt safety or less regulation after prompt
- Did a match convert to an official relationship tab state

10. Step-by-Step Build Plan for Developers

Build phase	Developer objective

Phase 1 - Canonical data foundation	Create tables, enums, indices, event catalog, schema versions, and write endpoints. Nothing fancy yet. Goal: stop ambiguity.
Phase 2 - Event instrumentation	Wire all front-end and backend behaviors to emit canonical events. Goal: the app starts remembering reality.
Phase 3 - State and trigger capture UX	Build daily check-in, post-chat check-in, conflict reflection, silence prompts, and trigger logging flows.
Phase 4 - Aggregation layer	Implement workers and materialized views that compute behavior signatures, self-awareness deltas, trigger summaries, and longitudinal trends.
Phase 5 - Packet builder	Create packet service that composes normalized AI-ready payloads with completeness and freshness scoring.
Phase 6 - AI insight service	Generate structured insight JSON from packet. Store narrative as optional display layer, never as the only output.
Phase 7 - Forecast service	Launch rule-based forecaster first, produce 7/30/90-day records, track outcomes.
Phase 8 - Intervention loop	Display prompts, rituals, couple reflections, and repair suggestions. Log outcomes.
Phase 9 - Drift + growth engine	Snapshot users monthly and after major events; show score movement, trigger changes, and archetype/shadow drift.
Phase 10 - QA + calibration	Compare forecast vs outcome, AI insight vs human review, and event completeness vs expected coverage.

Implementation order inside each phase:

- Define schema and enums.
- Add migrations and rollback scripts.
- Create endpoint contract and validation layer.
- Implement write path with idempotency and audit.
- Build background workers for aggregation.
- Expose read endpoints and admin debugging endpoints.
- Write unit tests for each event type and aggregate computation.
- Write QA test cases for consent boundaries and edge cases.
- Instrument dashboards for ingestion volume, failures, stale jobs, and drift.

11. Edge Cases and Failure Handling

Edge case	Required behavior
Partial consent	If one user has not consented to chat analysis, disable AI reading of shared chat content and fall back to self-report and app events only.
Missing data	If packet completeness is below threshold, forecast service should still run in degraded mode with lower confidence and explicit missing-input codes.
Duplicate events	Protect ingestion with idempotency keys and dedupe rules by actor, timestamp, and event fingerprint.
Late-arriving events	Allow aggregate recomputation when delayed events arrive out of order.
AI outage	Persist packet build request, queue retry, and keep product experience alive using deterministic non-AI rules.
Schema evolution	Version every event and packet schema so clients and services can coexist during rollout.
False trigger inference	Never treat AI-inferred trigger as equivalent to user-confirmed trigger without a confidence flag.
Conflict misclassification	Keep human-review tooling for samples and a manual override status on AI-classified conflict episodes.
Relationship breakup	Freeze prior relationship aggregates and create a closure snapshot rather than mutating the old relationship record.
Assessment retake abuse	Throttle retakes by policy and mark snapshots by retake reason and blackout exceptions.

12. Metrics to Track and Log

Metric	What it indicates
Packet completeness score	Percent of required blocks present
State freshness	Hours since last emotional state
Behavior coverage ratio	Observed events vs expected eligible events
Self-awareness delta	Difference between self-report and objective

	pattern
Volatility index	Swing magnitude across emotional and interaction data
Repair latency	Median time from rupture marker to repair attempt
Trigger recurrence rate	Frequency of repeated trigger classes
Forecast accuracy	Precision/recall or calibration by forecast type
Intervention efficacy	Outcome change after prompt or ritual
Drift score	Change in archetype/shadow/behavior signature over time

13. Final Development Recommendations

- Do not start with black-box machine learning. Start with clean events, exact schemas, and observable rule-based forecasting. Dirty input makes fake intelligence.
- Do not let front-end convenience dictate your data model. Build canonical backend truth first, then adapt UI to it.
- Do not store AI insight only as text. Require structured JSON output for every major AI call.
- Do not mix current profile state with historical records. Always snapshot.
- Do not treat onboarding answers as final truth. Behavior and longitudinal change must be able to update confidence over time.
- Do build internal admin tools: packet viewer, event browser, forecast inspector, and AI insight diff viewer. Without those, debugging will be a circus with Wi-Fi.

Bottom line:

Once the systems in this document exist, Relationship Scores will stop being a one-time assessment product and become a full behavioral intelligence platform. At that point the app is not just scoring love. It is reading emotional movement, tracking relational patterns, identifying distortions, forecasting outcomes, and learning from the truth of human behavior over time.

Appendix A - Recommended Enums

- context_type: daily_checkin, pre_chat, post_chat, post_conflict, post_silence, post_date, post_ritual
- event_type: message_sent, conversation_started, first_reply_after_gap, withdrawal_start, withdrawal_end, escalation_start, escalation_end, apology, repair_attempt, reassurance_request, reconnection
- trigger_type: delay, ambiguity, criticism, mixed_signal, pressure, vulnerability, perceived_replacement, unmet_expectation, conflict, withdrawal
- source: system_observed, user_reported, prompted_checkin, ai_classified, ai_inferred
- forecast_type: conflict_risk, silence_decay, repair_probability, rupture_probability, growth_probability, attachment_activation

Appendix B - Minimum Indexes

- user_emotional_state(user_id, state_at desc)
- behavior_event(relationship_id, event_at desc)
- behavior_event(conversation_id, sequence_index)
- perception_check(user_id, submitted_at desc)
- trigger_activation(user_id, trigger_at desc)
- profile_snapshot(user_id, effective_at desc)
- forecast_record(subject_type, subject_id, created_at desc)